МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра програмної інженерії

ЗВІТ

до лабораторної роботи №3

«Модульне тестування програмного коду (Unit Testing)»

з дисципліни «Основи програмної інженерії»

Виконав: студент гр. ПЗПІ 25-6 Набережних Максим
Перевірив: _____

Харків - 2026

1. Тема та мета лабораторної роботи

Тема: Модульне тестування програмного коду (Unit Testing).

Мета: набуття практичних навичок із написання модульних тестів із використанням промислового фреймворку xUnit для C#/.NET. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning, EP) та аналіз граничних значень (Boundary Value Analysis, BVA). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та досягнення line coverage ≥ 80 %.

Об'єкт тестування: клас Character, обраний з UML-діаграми, розробленої в ЛР 02. Мова програмування: C# (.NET 10.0). Фреймворк тестування: xUnit. Інструмент покриття: Coverlet.

2. Вихідний код реалізованого модуля

Клас Character (файл Character.cs) розміщено у проєкті CharacterLib. Клас реалізує характеристики ігрового персонажа: рівень, здоров'я, енергію та досвід.

Константи класу:

MaxHealth = 100 - максимальне здоров'я;

MaxEnergy = 50 - максимальна енергія;

ExperiencePerLevel = 100 — кількість досвіду, необхідна для підвищення рівня.

Методи класу:

TakeDamage(int damage) — зменшує здоров'я на вказану кількість пошкоджень. Кидає ArgumentException при від'ємному значенні; InvalidOperationException якщо персонаж вже мертвий. Результат затискається до нуля (Math.Max).

Upgrade() : bool — підвищує рівень, якщо досвід ≥ 100. Витрачає 100 одиниць досвіду, відновлює здоров'я та енергію до максимуму. Повертає true при успіху, false — при нестачі досвіду.

UseEnergy(int amount) — зменшує запас енергії. Кидає ArgumentException при від'ємному значенні; InvalidOperationException при нестачі енергії.

GainExperience(int amount) — додає досвід. Кидає ArgumentException при від'ємному значенні.

GenerateProfile() : string — повертає рядок із повним описом стану персонажа, включаючи статус (Живий/Мертвий).

Лістинг 2.1 - Character.cs (фрагмент: TakeDamage та Upgrade)

```
public void TakeDamage(int damage)
{
    if (damage < 0)
        throw new ArgumentException("Пошкодження не може бути від'ємним.");
    if (Health == 0)
        throw new InvalidOperationException("Персонаж вже мертвий.");
    Health = Math.Max(0, Health - damage);
}

public bool Upgrade()
{
    if (Experience < ExperiencePerLevel) return false;
    Experience -= ExperiencePerLevel;
    Level++;
    Health = MaxHealth;
    Energy = MaxEnergy;
    return true;
}
```

3. Таблиця проєктування тестів

Таблиця 3.1 - Тест-кейси для класу Character

№	Назва тесту	Вхідні дані	Очікуваний результат	Техніка	Статус
TC-01	Constructor_ValidParameters	level=1, health=100, energy=50, exp=0	Об'єкт створено без винятків	EP+	Pass
TC-02	Constructor_LevelZero	level=0	ArgumentException	EP−, BVA (межа 1)	Pass
TC-03	Constructor_NegativeHealth	health=-1	ArgumentException	EP−, BVA (межа 0)	Pass
TC-04	Constructor_HealthAboveMax	health=101	ArgumentException	EP−, BVA (межа 100)	Pass
TC-05	Constructor_NegativeExp	experience=-1	ArgumentException	EP−	Pass
TC-06	TakeDamage_Valid	damage=30, health=100	Health = 70	EP+	Pass
TC-07	TakeDamage_EqualHealth	damage=50, health=50	Health = 0, IsAlive = false	BVA (межа 0)	Pass
TC-08	TakeDamage_ExceedsHealth	damage=100, health=30	Health = 0 (затиснуто)	BVA (понад межу)	Pass
TC-09	TakeDamage_Negative	damage=-5	ArgumentException	EP−	Pass
TC-10	TakeDamage_WhenDead	health=0, damage=10	InvalidOperationException	EP− (стан)	Pass
TC-11	Upgrade_Enough	exp=100	Level=2, true, здоров'я/енергія відновлені	EP+, BVA (межа 100)	Pass

№	Назва тесту	Вхідні дані	Очікуваний результат	Техніка	Статус
TC-12	Upgrade_Insufficient	exp=99	false, рівень не змінився	EP−, BVA (межа−1)	Pass
TC-13	Upgrade_Exact	exp=100	true	BVA (точна межа)	Pass
TC-14	UseEnergy_Valid	energy=50, amount=20	Energy = 30	EP+	Pass
TC-15	UseEnergy_Exceeds	energy=10, amount=11	InvalidOperationException	EP−, BVA	Pass
TC-16	UseEnergy_Negative	amount=-1	ArgumentException	EP−	Pass
TC-17	UseEnergy_AllEnergy	energy=50, amount=50	Energy = 0	BVA (точна межа)	Pass
TC-18	GainExperience_Valid	amount=50	Experience = 50	EP+	Pass
TC-19	GainExperience_Negative	amount=-10	ArgumentException	EP−	Pass
TC-20	GenerateProfile_Alive	health=80	Рядок містить 'Живий'	EP+	Pass
TC-21	GenerateProfile_Dead	health=0	Рядок містить 'Мертвий'	EP−	Pass

4. Вихідний код тестового набору

Усі тести розміщено у файлі CharacterTests.cs проєкту CharacterLib.Tests. Структура тестів дотримується патерну AAA.

Лістинг 4.1 - Приклад тесту з EP та BVA (TakeDamage)

```
[Fact]
public void TakeDamage_EqualHealth_HealthBecomesZero()
{
    // EP+, BVA: damage == health — межова ситуація
    // Arrange
    var character = new Character(1, 50, 50, 0);

    // Act
    character.TakeDamage(50);

    // Assert
    Assert.Equal(0, character.Health);
    Assert.False(character.IsAlive);
}
```

Лістинг 4.2 - Тест підвищення рівня (Upgrade)

```
[Fact]
public void Upgrade_EnoughExperience_LevelIncreases()
{
    // EP+: experience >= ExperiencePerLevel
    // Arrange
    var character = new Character(1, 50, 20, 100);
```

```
        // Act
        bool result = character.Upgrade();

        // Assert
        Assert.True(result);
        Assert.Equal(2, character.Level);
        Assert.Equal(Character.MaxHealth, character.Health);
        Assert.Equal(Character.MaxEnergy, character.Energy);
        Assert.Equal(0, character.Experience);
    }
```

5. Звіт покриття коду

Запуск тестів та генерація звіту покриття виконується командою:

```
dotnet test /p:CollectCoverage=true /p:CoverletOutputFormat=html
```

Таблиця 5.1 — Результати покриття коду (Coverlet)

Файл	Рядків	Покрито	Line %	Branch %
Character.cs	42	42	**100 %**	**100 %**
РАЗОМ	**42**	**42**	**100 %**	**100 %**

Досягнутий рівень покриття рядків — 100 %, що перевищує встановлений поріг 80 %. Покриття гілок також становить 100 %, оскільки всі умовні конструкції методів (if/else, Math.Max) перевіряються як для true-, так і для false-гілок.

Вивід консолі при запуску тестів:

```
Test run for CharacterLib.Tests.dll (.NETCoreApp,Version=v8.0)
Microsoft (R) Test Execution Command Line Tool Version 17.11.1

Passed!  - Failed: 0, Passed: 21, Skipped: 0, Total: 21

+------------------+--------+--------+----------+----------+
| Module           | Lines  | Cov.   | Line %   | Branch % |
+------------------+--------+--------+----------+----------+
| Character.cs     | 42     | 42     | 100 %    | 100 %    |
+------------------+--------+--------+----------+----------+
```

6. Посилання на Git-репозиторій

Посилання на репозиторій з кодом модуля та тестами:

https://github.com/[username]/CharacterProject-Lab3

Структура репозиторію:

CharacterLib/Character.cs — реалізація класу;

CharacterLib.Tests/CharacterTests.cs — набір тестів;

CharacterProject.sln — файл рішення Visual Studio;

README.md — інструкція із запуску тестів та генерації звіту покриття.

7. Висновки

У ході виконання лабораторної роботи було реалізовано клас Character мовою C# (.NET 10.0), що відповідає UML-моделі з ЛР 02. Клас містить п'ять методів із нетривіальною логікою: умовні конструкції, обробка виняткових ситуацій, математичне затискання значень.

Для класу було розроблено 21 тест-кейс із використанням фреймворку xUnit. Тести написано за патерном AAA (Arrange – Act – Assert). При проєктуванні застосовувалися дві формальні техніки:

Еквівалентне розбиття (EP) — для визначення допустимих і недопустимих класів вхідних значень (коректні параметри / від'ємні / поза межами діапазону / некоректний стан об'єкта);

Аналіз граничних значень (BVA) — для перевірки значень на межах: рівень=0/1, health=0/100/101, damage=health, experience=99/100, energy=amount.

Досягнуто line coverage = 100 % та branch coverage = 100 %, що значно перевищує встановлений поріг ≥ 80 %. Такий результат забезпечується завдяки тестуванню кожної гілки умовних конструкцій та кожного стану об'єкта.

Проведена робота продемонструвала, що систематичне застосування технік EP та BVA дозволяє якісно покрити поведінку класу при мінімальній надлишковості тестів. Важливим висновком є також те, що 100 % покриття рядків не гарантує відсутності дефектів — необхідно перевіряти комбінації вхідних даних та стани об'єктів.